

Prince Sultan University

College of Computer & Information Sciences

Information Systems Department

IS499 – Senior Project

D4 – High-Level and Low-Level Design

Mirsad (مرصاد)

A Runtime Security Firewall for AI Agents Against Prompt-Injection Attacks

Team member	Student ID	Department
Mohammed Yasser Alanssari	221110935	IS (CYS)
Bader Alshuraie	221110016	SE
Bader Abdullah Abbod Alyamini	221110197	IS (CYS)
Abdulmalik bin Saedan	221110958	IS (CYS)
Abdullah Naif Abdullah Al-Orabi	222110432	SE (AI & DS)

Supervisor Dr. Omaia Al-Omari

Semester & Academic Year Senior Project I & II – Academic Year 2026–2027

Submission Date _____

Supervisor's Signature _____

Approval Date _____

Table of Contents

Table of Contents i
List of Figures ii
List of Tables iv
Software Design Document 1
3.3 System Design & Architecture 1
3.3.1 Architectural style and system context 1
3.3.2 Components and responsibilities 4
3.3.3 Core processes 7
3.3.4 Policy language design 13
3.3.5 Taint-tracking design 16
3.3.6 Injection detector design 18
3.3.7 Database design 19
3.3.8 Security design and threat model 22
3.3.9 Interface design 24
3.3.10 User-interface design 26
3.3.11 Deployment architecture and technology stack 30
References 32

List of Figures

Figure 1: System context: Mirsad sits between the AI agent and the enterprise tools; the attacker influences only the content stored in those tools 2

Figure 2: Mirsad architecture: protocol adapters, the call path (policy engine, rate limiter, taint tracker, combining step, human approval), the result path (injection detector, sanitising and spotlighting, taint labelling) and the shared services 3

Figure 3: Component (package) diagram. Dashed "injected" arrows are runtime collaborators passed to the gateway's constructor; every package also uses the foundation modules 5

Figure 4: Class diagram of the gateway, its collaborating services and the shared data model (signatures simplified; the full interfaces are in the engineering contract) 7

Figure 5: Sequence of a tool call through the gateway: decision, execution and inspection of the result 9

Figure 6: Decision logic for a tool call. Constraints, rate limits and global rules all report their reasons before the combining step 10

Figure 7: Sequence of an escalated call: approval request, notification, human decision and resumption of the waiting call 11

Figure 8: State machine of an approval request; every terminal state other than Approved blocks the call 12

Figure 9: Context-level (level 0) data-flow diagram of Mirsad 13

Figure 10: Level 1 data-flow diagram: processes 1.0 to 6.0 and data stores D1 to D4 13

Figure 11: Entity-relationship diagram of the six tables. The solid line is the only enforced foreign key; dashed lines are logical references by value 20

Figure 12: Threat model with trust boundaries A to E and the STRIDE categories of each data flow 22

Figure 13: Activity and audit page with the detail drawer of an approval decision: summary, previous hash and event hash, and the raw event 28

Figure 14: Policy editor in Arabic: the sidebar and controls are mirrored, while the YAML policy stays left-to-right 28

Figure 15: Approver PWA in Arabic: pending requests with risk label, countdown and plain-language summary 29

Figure 16: Approver PWA request detail in English: risk, countdown, explanation sections and the Approve and Deny buttons 30

Figure 17: Deployment architecture: the Mirsad container, the database, optional components and the clients 31

List of Tables

Table 1: Main design decisions 3
Table 2: Components, responsibilities and interfaces 5
Table 3: Structure of a policy document (schema version 1) 14
Table 4: Reason codes of the policy, detector, approval and system layers 15
Table 5: Taint rules evaluated for each tool call 17
Table 6: Data dictionary of the database tables 21
Table 7: STRIDE analysis of the gateway 22
Table 8: Agent-facing interfaces 25
Table 9: REST API overview 25

Software Design Document

This document is deliverable D4 (high-level and low-level design) of the Mirsad senior project. It describes the architecture, components, processes, data design, security design and user-interface design of the system as built. Requirements are specified in the separate D3 Software Requirements Specification.

This document is an extract of the Mirsad Final Report, where its content appears as Section 3.3. Section, figure and table numbers, and references to other sections, chapters and appendices, follow that report.

3.3 System Design & Architecture

This section defines the design of Mirsad as built: its architecture and context, components, core processes, policy language, taint tracking, injection detector, database, security design, interfaces, user interface and deployment. The design follows the engineering contract `docs/dev/BUILD_SPEC.md` (Section 3.4.1), and every diagram was drawn from, or checked against, the code in `src/mirsad/`; the Mermaid sources are kept in `docs/diagrams/`.

Four protection principles [1] guided the design:

- **Complete mediation.** Every tool call and every tool result passes through one enforcement point.
- **Fail-safe defaults.** Unknown agents and undeclared tools are denied, and an internal error, an unavailable audit log or an unanswered approval request ends in denial.
- **Least privilege.** Permissions are granted per agent, per tool and per argument, and an agent only discovers the tools its policy allows.
- **Separation of privilege.** High-risk actions need a human approver, and the auditor role can review the audit log without being able to change policies.

3.3.1 Architectural style and system context

Mirsad is a **policy enforcement point (PEP)** in the sense of the XACML reference architecture [2]. The gateway (`mirsad.gateway.Gateway`) is the PEP and the `PolicyEngine` is the policy decision point; the taint tracker and the injection detector act as attribute providers that supply the provenance of the session's data and evidence about tool results; human approval is an obligation that must be discharged before an escalated call proceeds; and the audit log records every decision. Unlike a conventional PEP, Mirsad mediates in two directions. The **call path** checks what the agent wants to do before the tool runs, and the **result path** inspects what the tool returned before the model reads it, because indirect prompt injection arrives inside tool results [3].

Placing the enforcement point at the tool-calling boundary, rather than inside the model or the agent's code, was the central architectural decision. It makes Mirsad independent of the model (NFR2), requires no change to the agent apart from its tool endpoint, and governs actions rather than text. Designs that restructure the agent, such as CaMeL [4] and the patterns of Beurer-Kellner et al. [5], give stronger guarantees but cannot be added to an existing agent. Privilege-control systems such as Progent [6] are closest to Mirsad's policy layer; Mirsad adds data-flow tracking, Arabic-aware detection, human approval and a verifiable audit trail around such a layer.

Figure 1 shows the system context. The agent calls enterprise tools through Mirsad on behalf of a business user; the attacker can only place content in systems the agent reads, such as e-mail, web pages and documents; approvers decide on their phones; and administrators and auditors work in the dashboard.

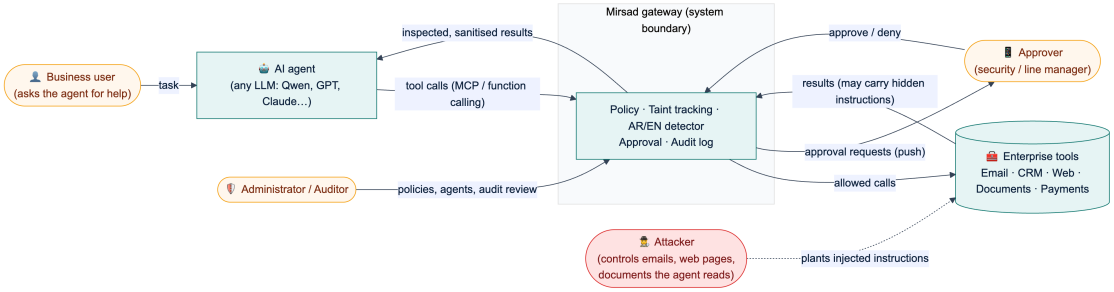


Figure 1: System context: Mirsad sits between the AI agent and the enterprise tools; the attacker influences only the content stored in those tools

Figure 2 shows the internal architecture, which combines four styles:

1. **Layers.** Protocol adapters (MCP proxy, OpenAI-compatible proxy, Guard API) translate external protocols into one internal model (`ToolCall`, `ToolResult`); the gateway core orchestrates the defence layers; storage sits underneath. A new agent protocol only touches the adapter layer.
2. **Pipes and filters.** On the call path, the policy engine, rate limiter and taint tracker each examine the call and emit `Reason` objects, which a combining step reduces to one decision. On the result path, detection, sanitising or spotlighting, and taint registration run in sequence.
3. **Publish–subscribe.** Approval events and new audit events are published on an in-process notifier and delivered over Server-Sent Events (SSE) and, optionally, Web Push.
4. **Modular monolith.** All components run in one Python process behind one FastAPI application. For a five-person team and a 200 ms latency budget (NFR1), in-process calls were preferred over separate services; typed interfaces and constructor

injection (Section 3.3.2) keep the module boundaries, so a layer can later be moved out of process.

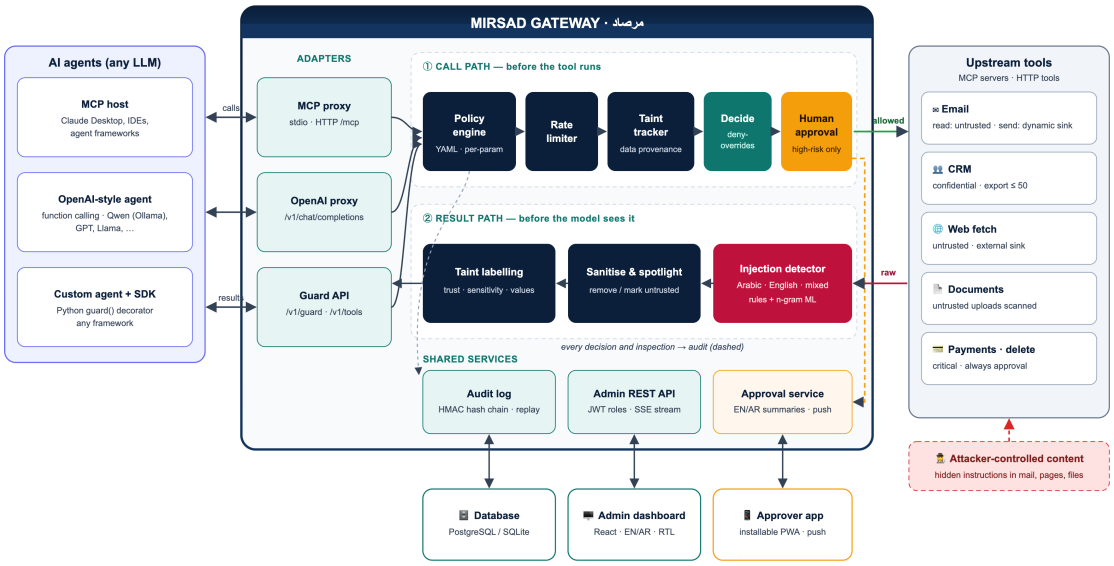


Figure 2: Mirsad architecture: protocol adapters, the call path (policy engine, rate limiter, taint tracker, combining step, human approval), the result path (injection detector, sanitising and spotlighting, taint labelling) and the shared services

Table 1 records the main design decisions, the alternatives considered and the reasons for each choice.

Table 1: Main design decisions

Decision	Alternatives considered	Rationale
Enforce at the tool boundary as a proxy	Guardrail at the model boundary; rebuild the agent around a secure architecture	Works with any LLM and agent; governs actions; inline adapters give complete mediation
Deterministic policy and taint rules as the primary control, the detector as a supporting signal	Rely mainly on an injection classifier	Published detectors are bypassed by adaptive attacks [7]; rules on actions and data flows still hold when text detection misses
Fixed deny-overrides combination with three outcomes	Configurable combining algorithms; first-applicable	Adding a layer or a rule can only make a decision stricter, so independent layers compose safely
Fail closed in every component	Fail open to preserve availability	A control that silently passes calls when it breaks gives false assurance (NFR6)
Template-based bilingual approver summaries	Summaries written by an LLM	A summary built from reason codes cannot be prompt-injected and is reproducible
HMAC hash chain inside the relational database, with optional external anchors	Separate log service; Merkle-tree log; distributed ledger	No extra infrastructure; verifiable offline from an export; a Merkle tree [8] remains an upgrade path

Decision	Alternatives considered	Rationale
Store each decision with its inputs (policy version, minimal taint snapshot, rate counters)	Store only the outcome	Any decision can be replayed deterministically (FR6.3)
One process with constructor injection; SQLite or PostgreSQL	Microservices; Redis for shared state	Fits the team size and latency budget; injection lets tests and the benchmark swap layers

3.3.2 Components and responsibilities

Figure 3 shows the Python packages in `src/mirsad/`, the clients outside the gateway and their dependencies, derived from the import graph of the code; an arrow points from a package to the package it uses. Two dependency rules hold. First, the defence layers (`policy`, `taint`, `detector`) depend only on the shared data model (`mirsad.models`), the text utilities (`mirsad.textutil`) and, for `taint`, the policy's settings types, so each can be tested and benchmarked on its own. Second, the gateway does not construct its collaborators: at start-up `api/components.py` builds the object graph and passes the layers to the `Gateway` constructor, which imports them for type checking only (dashed "injected" arrows). Any component can be replaced through `create_app(gateway_overrides=...)`, which the test suite and the Mirsad-Bench ablations (policy only, taint only, detector only) rely on.

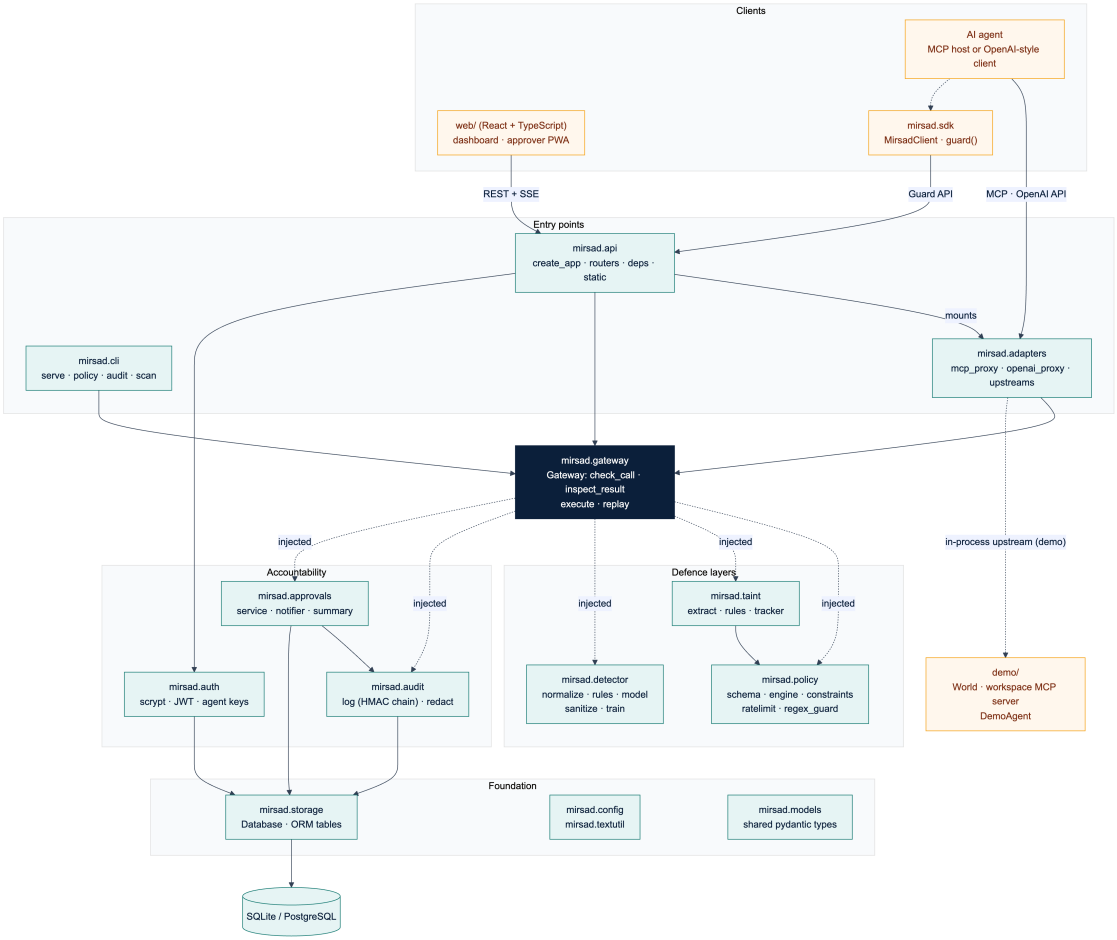


Figure 3: Component (package) diagram. Dashed "injected" arrows are runtime collaborators passed to the gateway's constructor; every package also uses the foundation modules

Table 2 lists each component's responsibility, main interface and owner (BS = Bader Alshuraie, AO = Abdullah Naif Abdullah Al-Orabi, BY = Bader Abdullah Abbod Alyamini, MA = Mohammed Yasser Alanssari, AB = Abdulmalik bin Saedan).

Table 2: Components, responsibilities and interfaces

Component	Responsibility	Main interface	Owner
`adapters`	Translate MCP (stdio, streamable HTTP) and OpenAI chat completions into the internal model; connect to upstream MCP servers and Python tools	`build_mcp_server`, `build_openai_router`, `UpstreamManager`	BS
`gateway`	Decision pipeline: check calls, inspect results, execute allowed calls, replay stored decisions; fail closed	`Gateway.check_call`, `inspect_result`, `execute`, `replay`	BS
`policy`	Parse and validate YAML policies; resolve tool profiles and sinks; evaluate constraints, rate limits and rules	`load_policy`, `validate_policy`, `PolicyEngine`, `RateLimiter`	BS

Component	Responsibility	Main interface	Owner
`taint`	Label results with trust and sensitivity, extract values with their provenance, evaluate the four data-flow rules	`TaintTracker`, `extract_values`	BY
`detector`	Normalise Arabic and English text, score it with rules and a linear model, sanitise and spotlight	`InjectionDetector.scan`, `sanitize`, `spotlight`	AO
`audit`	Redact, chain, store, verify, query and export audit events	`AuditLog`, `verify_events`, `redact`	BY
`approvals`	Approval workflow, bilingual summaries, SSE publish–subscribe and Web Push	`ApprovalService`, `Notifier`, `summarize`	BS, BY (UX: MA)
`auth`	Password hashing, JWTs, agent API keys, user and agent management	`hash_password`, `create_token`, `authenticate_agent_key`	BY
`storage`	Database engine, sessions and ORM tables	`Database`, table classes	BY
`api`	Application factory, routers, authentication dependencies, security middleware, static hosting	`create_app`, `require_user`, `require_agent`	BS
`sdk`	Python client and `guard()` decorator for frameworks that execute tools themselves	`MirsadClient`, `AsyncMirsadClient`	BS
`cli`	Serve, initialise, manage policies, verify, export and replay the audit log, scan text, create agents and users, stdio MCP proxy	`mirsad ...` commands	BS
`web/`	Administration dashboard and approver PWA	REST and SSE client	MA
`demo/`, `bench/`	Simulated Najd Trading Co. workspace and demonstration agent; Mirsad-Bench harness	`World`, `DemoAgent`, `bench.run`	AB

Figure 4 shows the main classes and the shared data model. `Gateway` aggregates five runtime services; `PolicyEngine` owns its `RateLimiter`; `ApprovalService` publishes through a `Notifier`; and `UpstreamManager.call` is passed to `Gateway.execute` as the executor of allowed calls, so the gateway never needs to know how a tool is reached. The data classes (stereotype «model») are Pydantic models defined once in `mirsad/models.py`. `ToolCall` and `ToolResult` are the protocol-independent forms of a call and its result. `ToolProfile` is the security profile of one tool for one agent (allowed, trust, sensitivity, sink, risk, approval mode). A `Reason` is one piece of evidence from one layer: a machine-readable code, English and Arabic messages, a proposed `Action`, a risk score and structured evidence. A `Decision` combines the reasons with the risk score, sink, policy version, approval identifier and audit sequence number, and an `InspectionOutcome` is the verdict on a result (pass, flag, sanitize or block) with the content the model will see. `Action` is ordered (allow < require_approval < deny), and `Risk` maps a score to a level (medium from 0.3, high from 0.6, critical from 0.85). Because every layer reports in the same `Reason` type, evidence from different layers can be combined uniformly.

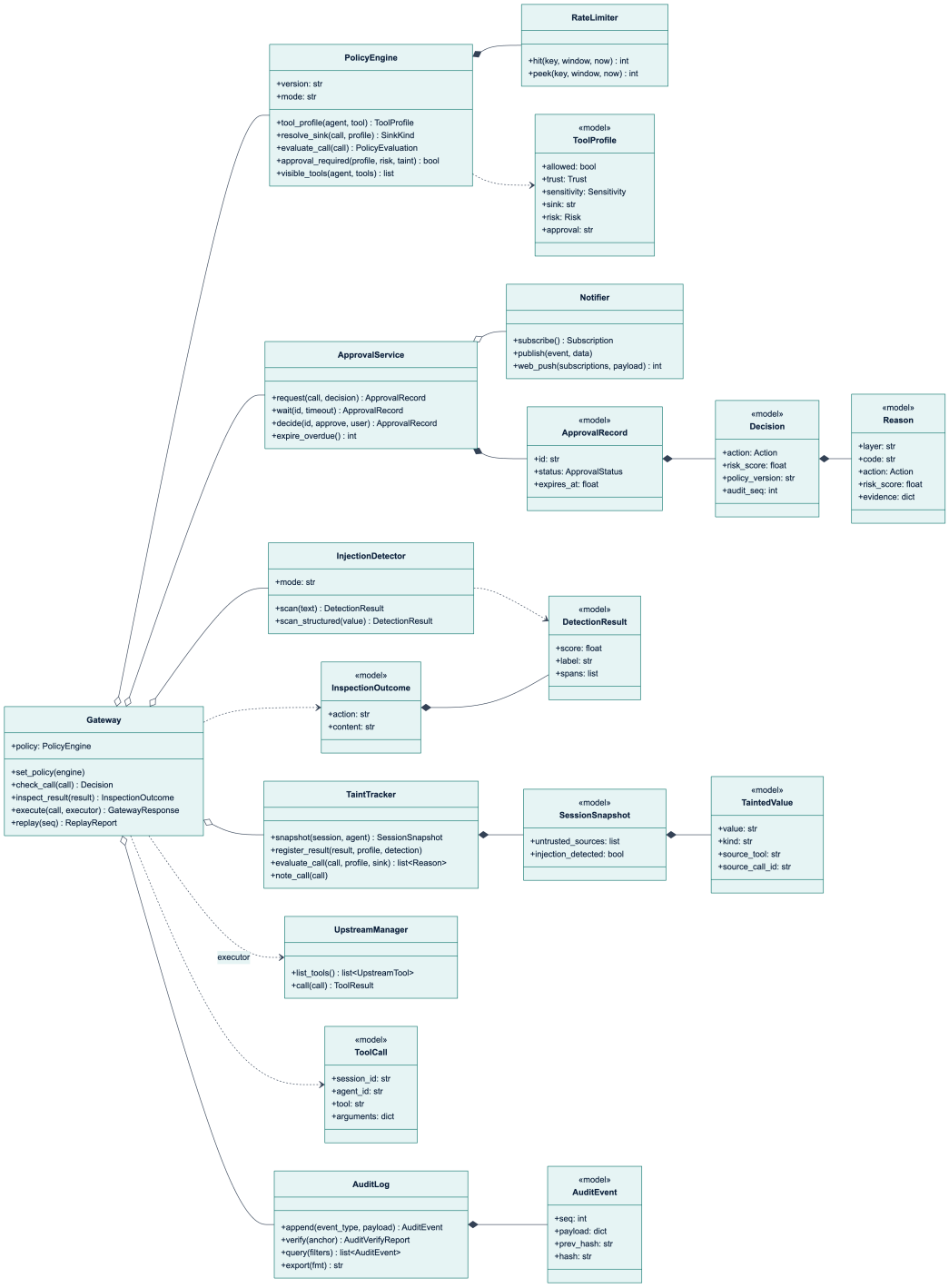


Figure 4: Class diagram of the gateway, its collaborating services and the shared data model (signatures simplified; the full interfaces are in the engineering contract)

3.3.3 Core processes

P1: deciding on a tool call. Figure 5 shows the sequence for a call arriving through any adapter, and Figure 6 shows the decision logic. `Gateway.check_call` performs these steps:

1. Read the session's taint snapshot and call `PolicyEngine.evaluate_call`, which resolves the tool profile and sink and returns the policy and rate-limit reasons (Section 3.3.4).
2. Unless the agent is unknown, call `TaintTracker.evaluate_call` with the resolved sink (Section 3.3.5).
3. Combine all reasons with deny-overrides: the most severe action wins and the risk score is the highest score.
4. If the result is ALLOW but the tool's approval mode requires a human at this risk level, add `APPROVAL_POLICY` and escalate to REQUIRE_APPROVAL.
5. Append a `tool_call.decision` audit event with the call, profile, sink, all reasons, the part of the taint snapshot the rules consulted, the rate-limit counters and the risk score.
6. If approval is required, request it and wait (process P3); the final action is ALLOW only if the request is approved.
7. Record the call in the session and return the `Decision`, whose `latency_ms` excludes time spent waiting for a human.

Steps 1 to 5 run in a worker thread under one re-entrant lock, so snapshot, evaluation and audit append are atomic with respect to concurrent calls while the event loop stays free. Three fail-closed rules apply: an exception in any layer gives DENY with `INTERNAL_ERROR`; a failed audit append gives DENY with `AUDIT_UNAVAILABLE`, because no action may happen without a record; and a failed approval service gives DENY with `APPROVAL_UNAVAILABLE`. In monitor mode the would-be decision is computed and audited with `enforced: false`, but ALLOW is returned and results are not altered, so a policy can be tuned in shadow deployment before it is enforced.

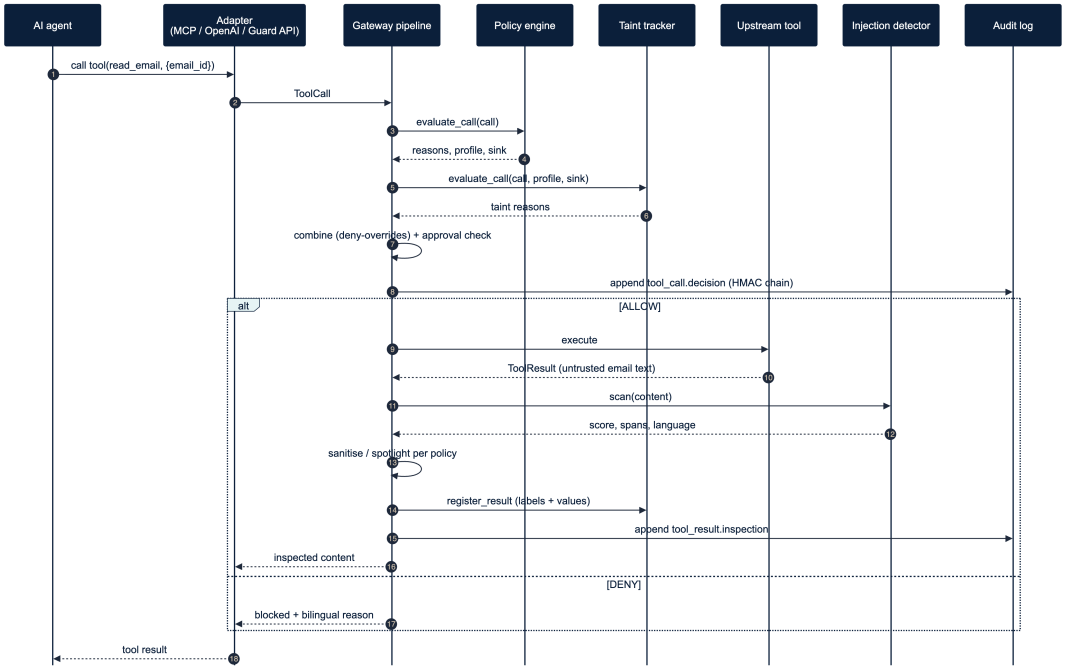


Figure 5: Sequence of a tool call through the gateway: decision, execution and inspection of the result

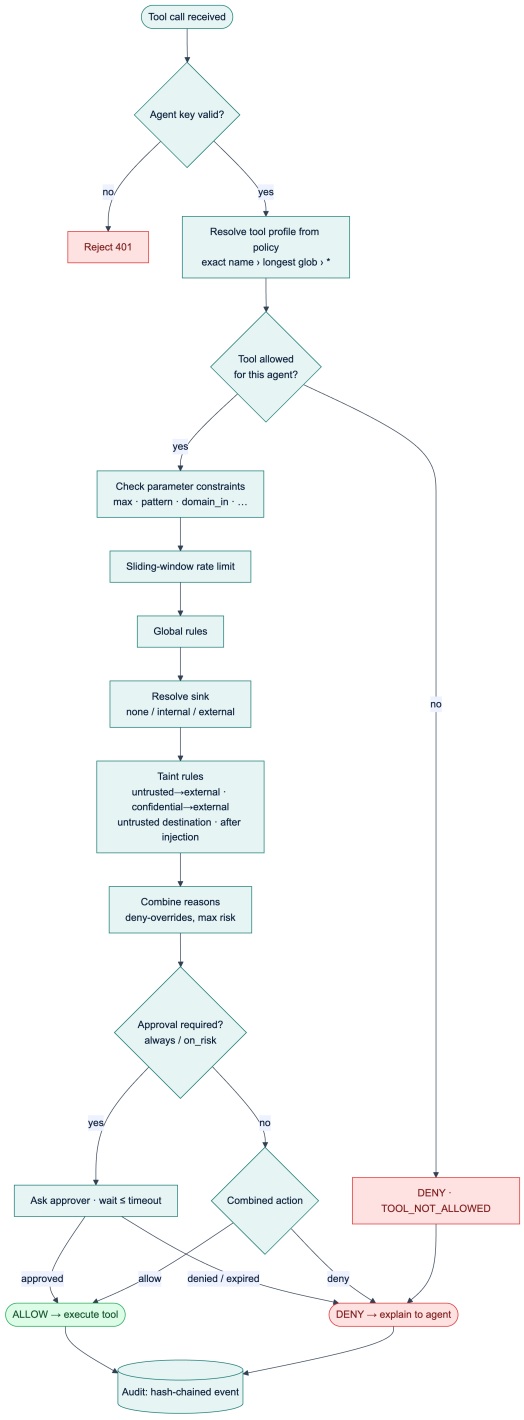


Figure 6: Decision logic for a tool call. Constraints, rate limits and global rules all report their reasons before the combining step

P2: inspecting a tool result. `Gateway.inspect_result` scans the content when the tool is marked `scan` or its results are untrusted (results of tools the policy does not allow are always untrusted), maps the score with the tool's thresholds to pass, flag, sanitize or

block, spotlights untrusted content that is not blocked, registers the result with the taint tracker and appends a `tool_result.inspection` event. A sanitiser that cannot locate the suspicious text escalates to block, and a detector failure withholds the result. `Gateway.execute` composes P1 and P2: it runs the tool (with a 120 s default timeout) only for allowed calls and re-binds the result to the original call and session, so that an upstream cannot attribute its output to another call.

P3: human approval. Figure 7 and Figure 8 show the approval process and its state machine. `ApprovalService.request` stores a pending record with the redacted arguments, the decision, English and Arabic summaries and an expiry time (120 s by default), publishes `approval.requested` over SSE and, when configured, Web Push. The waiting call blocks on an event object, with a database re-check every 5 s for decisions made by another process. Every transition is a conditional `UPDATE ... WHERE status = 'pending'`, so exactly one decision wins when two approvers act at once, and a second decision is rejected with HTTP 409. At the deadline the request becomes EXPIRED, which the gateway treats as DENY; this also prevents a late approval of an action the agent was already told had been refused.

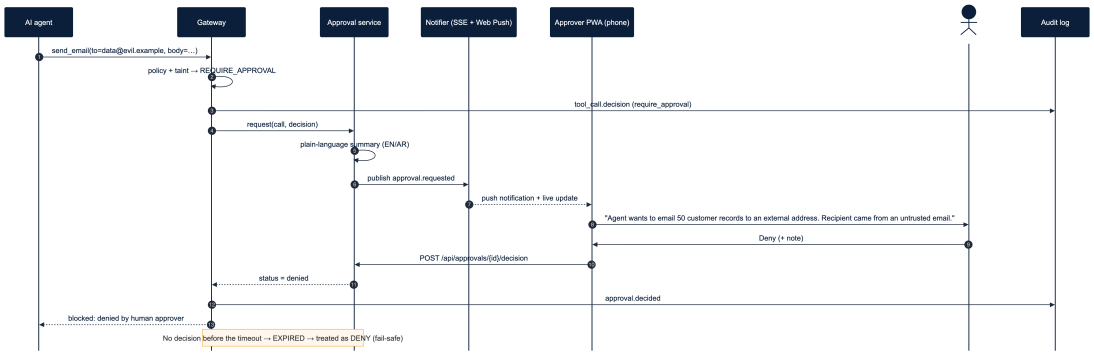


Figure 7: Sequence of an escalated call: approval request, notification, human decision and resumption of the waiting call

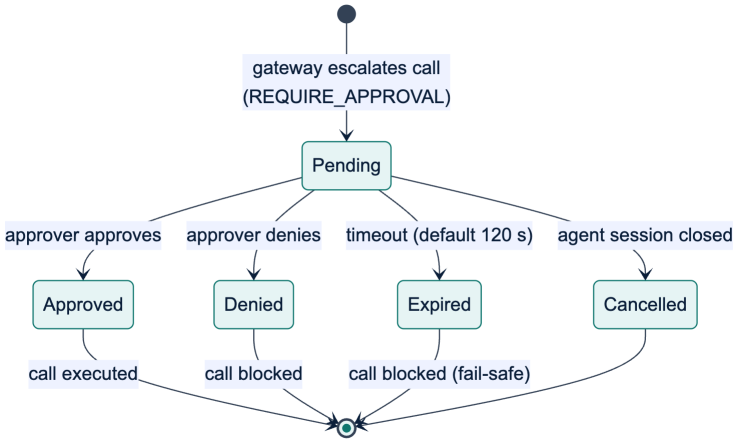


Figure 8: State machine of an approval request; every terminal state other than Approved blocks the call

The approver's summary is built by `approvals/summary.py` from the reason codes and their evidence, not by a language model, so attacker-controlled text cannot rewrite it; values copied from arguments are length-limited and stripped of line breaks. The summaries below were produced by this code for a benign case, in which an agent that has read an external supplier's e-mail replies to that supplier.

> The Email & CRM Assistant wants to send an email to accounts@red-sea-logistics.example (outside Najd Trading Co.) using the send_email tool. Mirsad paused it because the agent read an untrusted email (read_email call_r1) earlier in this session, and this action sends data outside the organisation, and the recipient address is outside the allowed domains (najd-trading.example). Risk level: high.

يريد الوكيل «Email & CRM Assistant» إرسال بريد إلكتروني إلى accounts@red-sea-logistics.example (خارج شركة نجد للتجارة) عبر الأداة send_email. أوقف مرصاد الطلب مؤقتًا لأن الوكيل اطّلع في هذه الجلسة على بريد إلكتروني غير موثوق (read_email call_r1)، وهذا الإجراء يرسل بيانات إلى خارج المؤسسة، ولأن عنوان المستلم خارج النطاقات المسموح بها (najd-trading.example). مستوى الخطورة: مرتفع.

The example shows a deliberate trade-off. The recipient came from the e-mail's `from` field, which the policy lists as trusted, so the rule that denies destinations chosen by untrusted text does not fire; the session-level taint rule and the domain constraint still ask a human to confirm. This is safe but adds friction to legitimate work.

P4: audit verification and replay. Verification recomputes the hash chain (Section 3.3.7). `Gateway.replay(seq)` loads the policy version recorded in a decision event, rebuilds the call and the stored taint snapshot, re-runs the policy with the stored rate counters (leaving the live limiter untouched) and the taint rules on the snapshot, and compares the action and reason codes; inspection events are rescanned. Because the taint

rules are a pure function of the call and the snapshot, replay is exact (Section 4.2.7 reports the replay results).

Data-flow view. Figure 9 is the context-level data-flow diagram, and Figure 10 decomposes the gateway into six processes: 1.0 authenticate and adapt, 2.0 evaluate the policy, 3.0 check taint, 4.0 obtain approval, 5.0 inspect the result and 6.0 audit. Data stores D1 (policies), D2 (audit events) and D3 (approvals) are database tables. D4 (session taint state) is kept in memory per process, bounded in size and expired after six idle hours; the part of D4 a decision depends on is copied into its audit event, so the audit log, not memory, is the durable record.

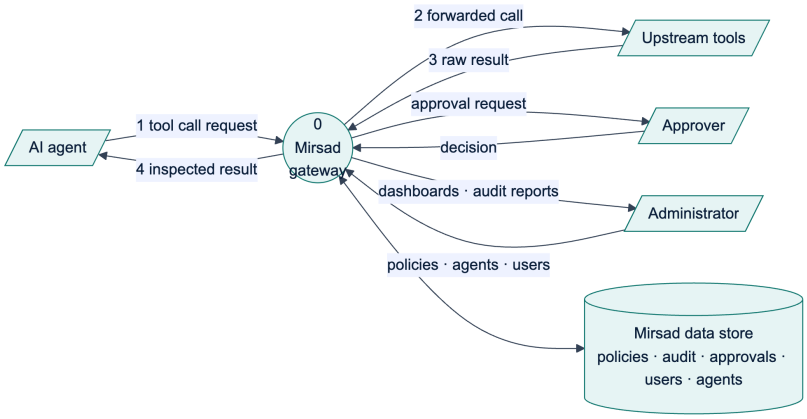


Figure 9: Context-level (level 0) data-flow diagram of Mirsad

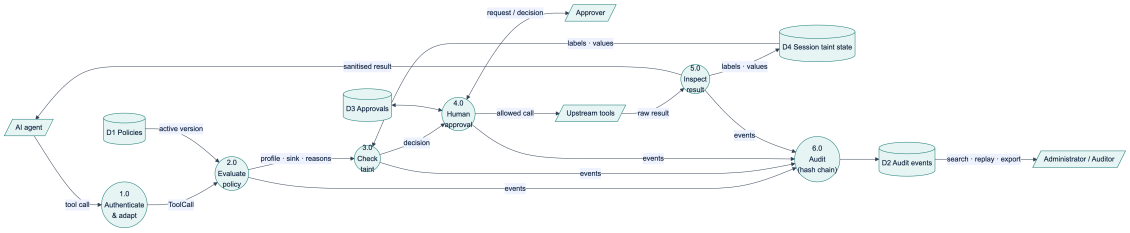


Figure 10: Level 1 data-flow diagram: processes 1.0 to 6.0 and data stores D1 to D4

3.3.4 Policy language design

The policy language had four goals: a policy must be reviewable by a security analyst in a pull request; permissions must default to deny and reach individual arguments; mistakes must be caught with precise messages before activation; and evaluation must be deterministic and versioned so that decisions can be replayed. General-purpose languages such as Rego [9] or XACML [2] are more expressive, but a domain-specific YAML format with a strict Pydantic schema fits a prompt-injection policy on one page and is harder to get dangerously wrong. Table 3 summarises the structure of a policy document.

Table 3: Structure of a policy document (schema version 1)

Element	Keys	Meaning
Document	`version`, `mode` (enforce or monitor), `organization` (`name`, `internal_domains`)	Schema version; shadow mode; domains inside the organisation
`defaults`	`decision` (deny, allow or require_approval), `approval_timeout_s`	Treatment of unmatched tools (deny by default); time a human has to decide
`detector`	`flag_threshold`, `block_threshold`, `on_detect` (sanitize, block or flag), `spotlight`	Global detector settings; agents and tools can override them
`taint`	`enabled`, `destination_params`, `trusted_fields`, one action per taint rule	Data-flow settings (Section 3.3.5); agents can override them
`agents[]`	`id`, `name`, `description`, `tools` (tool key → tool entry)	One entry per agent identity; `id` equals the agent's `policy_agent_id`
Tool entry	`allow`, `trust`, `sensitivity`, `sink` (none, internal, external or dynamic), `sink_params`, `risk`, `approval` (never, on_risk or always), `scan`, `constraints`, `rate_limit`, `detector`	Security profile of one tool for one agent
Constraint	`max`, `min`, `max_length`, `min_length`, `enum`/`in`, `not_in`, `pattern`, `not_pattern`, `domain_in`, `domain_not_in`, `max_items`, `required`, `type`, `on_violation`	Checks on one argument, combined with AND; a violation denies or requires approval
`rules[]`	`id`, `when` (`tool`, `agent`, `arg_matches`, `sink`, `session_tainted`, `min_risk`), `action`	Global rules evaluated after the tool entries

The excerpt below, from the tool entries of `configs/policies/default.yaml`, shows how the business rules of Section 3.2 are expressed: e-mail outside the organisation needs approval (BR4), exports are capped at 50 records (BR3), payments always need approval (BR2), and unlisted tools are denied (BR1).

```
send_email:
  allow: true
  sink: dynamic
  sink_params: [to, cc, bcc]
  risk: medium
  constraints:
    to:      {domain_in: [najd-trading.example], on_violation: require_approval}
    body:    {max_length: 20000}
  rate_limit: {max_calls: 20, per_seconds: 3600}
get_customer:     {allow: true, sensitivity: confidential, scan: true}
export_customers:
  allow: true
  sensitivity: confidential
  scan: true
  risk: high
  approval: on_risk
  constraints:
    count: {max: 50, on_violation: deny}
  rate_limit: {max_calls: 3, per_seconds: 3600}
transfer_funds:   {allow: true, risk: critical, approval: always, sink: external,
                   constraints: {amount_sar: {max: 10000, on_violation: deny}}}
"*":              {allow: false}
```

CRM results are trusted and confidential, but they are scanned (`scan: true` on `search_customers`, `get_customer` and `export_customers`), because their free-text notes can contain text that came from web forms or customer e-mails; a detection relabels the result as untrusted (Section 3.3.5).

Resolution. For a tool name the engine selects the exact key, otherwise the longest matching glob, otherwise the catch-all `"*"`, otherwise `defaults.decision`; equal-length globs that can match the same name are rejected as ambiguous. A `dynamic` sink is EXTERNAL if any recipient or URL in the destination arguments has a domain outside `internal_domains` or no recognisable domain, INTERNAL if all destinations are internal, and NONE if there is none.

Evaluation and combining algorithm. `PolicyEngine.evaluate_call` collects reasons in a fixed order: (1) an unknown agent is denied and evaluation stops; (2) a tool that is not allowed is denied and evaluation stops; (3) an informational `TOOL_RISK` reason carries the tool's risk score (0.1, 0.3, 0.6 or 0.9 for low to critical); then (4) every argument constraint, (5) the sliding-window rate limit, (6) every matching global rule and (7) `approval: always`. After step 3 nothing short-circuits, so the audit log and the approver see every objection. The gateway adds the taint reasons and applies **deny-overrides**: for the reasons r1, …, rn of a call, the final action is the most severe among them (deny > require_approval > allow) and the risk score is the maximum of their scores. The approval mode is applied last: `always` escalates every call; `on_risk` escalates when the combined risk is at least 0.6, or when the tool's own risk is high or critical and a taint rule objected; `never` does not escalate, although a deny still denies. Deny-overrides is monotonic, so adding a layer, constraint or rule can only make a decision stricter. The accepted limitation is that an `allow` rule cannot create an exception to a deny; exceptions are written as narrower deny rules. Table 4 lists the reason codes outside the taint layer, whose codes are given in Section 3.3.5.

Table 4: Reason codes of the policy, detector, approval and system layers

Code	Layer	Action	Risk score
`UNKNOWN_AGENT`, `TOOL_NOT_ALLOWED`	policy	deny	0.5
`TOOL_RISK`	policy	allow (informational)	0.1 / 0.3 / 0.6 / 0.9
`CONSTRAINT_VIOLATION`	policy	the constraint's `on_violation`	0.5
`RATE_LIMIT`	ratelimit	deny	0.5
`RULE_MATCH`	policy	the rule's `action`	rule's score, else 0.5 (0 for allow)
`APPROVAL_ALWAYS`	policy	require approval	the tool's risk score
`APPROVAL_POLICY`	approval	require approval	combined score
`APPROVED_BY_HUMAN`,	approval	allow, deny, deny	unchanged

Code	Layer	Action	Risk score
`DENIED_BY_HUMAN`, `APPROVAL_EXPIRED`			
`INTERNAL_ERROR`, `AUDIT_UNAVAILABLE`, `APPROVAL_UNAVAILABLE`	system, approval	deny (fail closed)	1.0
`MONITOR_MODE`	system	allow (records the would-be action)	0
`INJECTION_SUSPECTED`, `INJECTION_DETECTED`	detector	deny when the result is blocked; otherwise allow, with the result flagged or sanitised	detector score
`SCAN_FAILED`	detector	deny (result withheld, fail closed)	1.0
`RESULT_TRUNCATED`	system	allow (informational)	0

Safe validation. The schema is strict: unknown keys, operators and duplicate YAML keys are errors, with "did you mean" suggestions and the YAML line and column. Because policy regular expressions run against arguments written by a possibly hijacked model, the validator rejects patterns with nested ambiguous quantifiers, the classic cause of catastrophic backtracking (ReDoS); at run time at most 50,000 characters reach a policy regex, and a truncated input counts as a rule match, so the check fails closed.

Versioning and hot reload. A policy's version is the SHA-256 hash of the canonical JSON of the parsed document, so formatting does not change it but any semantic change does. Every version is stored, and the version in force is recorded in every audit event. Engines are immutable: activation swaps in a new engine atomically, and an evaluation in progress keeps the engine it started with.

3.3.5 Taint-tracking design

An LLM can mix any input into any output, so taint cannot be propagated through the model, and the guarantees of language-based information-flow control are out of reach [10]. Mirsad therefore tracks data at the boundary it controls, in the style of dynamic taint analysis [11]: tool results are **sources**, tool calls are **sinks**, and the rules look for the combination of private data, untrusted content and external communication that makes an agent exploitable [12]. The approach follows information-flow control for agents [13], [14], except that a violation can be routed to a human approver instead of always aborting.

Labels. Every result receives two labels from its tool profile, as in Denning's lattice model [15]: integrity (trusted or untrusted, where untrusted means a third party can control the content) and confidentiality (public < internal < confidential). Detector evidence overrides declared trust: a result labelled as an injection is registered as

untrusted even if its tool is declared trusted. Each recorded value also carries a `trusted_field` flag for fields vouched for by the producing system rather than the content's author, such as the sender, identifier and date of a received e-mail. This separates "reply to the sender", which is normal, from "send to an address written in the body", the typical exfiltration pattern.

Two granularities. At **session level**, a session that has read untrusted content stays tainted for the rest of its life, a conservative but sound assumption. At **value level**, recognisable values are extracted from every result with their source tool, call and field: e-mail addresses, URLs, domains, IBANs, phone numbers, numbers of at least six digits, record identifiers and, for confidential results only, hashed word 8-grams of long text. Results and arguments are canonicalised the same way (Unicode NFKC, invisible characters removed, Arabic-Indic digits converted, case folded, Saudi mobile numbers unified), so a value matches whatever its spelling. N-grams are hashed with BLAKE2b, which keeps them stable across processes for replay and avoids storing the text. Table 5 lists the four rules evaluated before every call.

Table 5: Taint rules evaluated for each tool call

Rule (reason code)	Fires when	Default action	Risk
`TAINT_UNTRUSTED_TO_EXTERNAL`	The session has read untrusted content and the call's sink is external	require approval	0.6
`TAINT_CONFIDENTIAL_TO_EXTERNAL`	An argument contains a value recorded from a confidential result and the sink is external (sending customers only their own record is exempt)	require approval	0.7
`TAINT_UNTRUSTED_DESTINATION`	A destination argument (recipient, URL, IBAN, webhook) or its domain first appeared in untrusted content outside a trusted field, and never in a trusted result; the organisation's own domains are excluded	deny	0.9
`TAINT_AFTER_INJECTION`	An injection was detected earlier in the session and the tool's risk is high or critical	require approval	0.8

Every taint reason names the source tool, call and field of the matching value, so the approver sees where the data came from. State is bounded (at most 2,000 values per session; sessions dropped after six idle hours), and the rules are one pure function of the call, profile, sink, settings and a read-only view of the session, used by both live evaluation and replay.

The coarse session-level rule is the price of soundness; Section 4.2.4 measures the approval friction it causes. Value-level matching, in turn, can be defeated by paraphrasing or re-encoding data, which is why the session-level rule and human approval remain as a backstop.

3.3.6 Injection detector design

The detector reads untrusted tool results and the results of tools marked `scan: true`, such as CRM records, whose free-text notes can contain text that came from outside the organisation. It is designed as a **signal, not a security boundary**, because adaptive attacks have bypassed published detectors with high success rates [7]; the policy and taint layers are built to block harmful actions even when the detector misses. The detector catches common cases cheaply, drives the result-path actions, arms `TAINT_AFTER_INJECTION` and gives approvers and auditors readable evidence. It works in six stages:

1. **Normalisation.** HTML entities are decoded; zero-width and bidirectional control characters removed; text converted with NFKC and lower-cased; Arabic diacritics (tashkeel) and elongation (tatweel) removed; alef, ya and ta marbuta variants and Persian look-alikes unified; digits converted to ASCII; Cyrillic, Greek and Armenian look-alikes in Latin words folded; and letter-spaced words joined. Obfuscation is reported as *signals*, and an offset map takes each span back to the original text, so highlights and sanitising point at the characters the user and the model see.
2. **Language identification** (English, Arabic, mixed or other, including Arabizi), for reporting and per-language evaluation only.
3. **Rules.** 158 weighted regular-expression rules in YAML: 76 English, 51 Arabic (Modern Standard Arabic and Gulf, with some Egyptian and Levantine forms), 7 Arabizi and 24 language-independent rules for structural tricks such as chat-template tokens, hidden HTML, markdown-image exfiltration and encoded blobs. The rule score combines the best weight of each distinct rule by noisy-OR, and evasion signals raise it only when a rule has fired, because ordinary Arabic text produces some of them.
4. **Model.** Hashed character 2- to 5-grams (2^20 features) and word 1- to 2-grams (2^18 features) feed a class-balanced logistic regression. Texts are scored over windows of 600 characters with a stride of 300 and the highest window probability is kept, so one injected sentence in a long e-mail is not diluted.
5. **Ensemble.** The score is a noisy-OR of model score m and rule score r, $1 - (1 - m)^{\alpha} (1 - r)^{\beta}$, calibrated so that the operating point chosen on the validation split (highest F1 with a false-positive rate of at most 5%) maps to 0.5; the policy defaults (flag at 0.5, block at 0.8) keep their meaning after retraining.
6. **Persistence without pickle.** The model is stored as plain arrays and JSON metadata, loaded with `allow_pickle=False` and checked for shape and finite values; if it is missing or invalid, the detector runs in rules-only mode instead of failing.

A linear model over hashed n-grams was chosen over a fine-tuned transformer because the detector runs inline on CPU, the team's dataset is small, and character n-grams

tolerate Arabic morphology, spelling variation and Arabizi (Section 3.4.5). On the result path, scores below the flag threshold pass, scores between the thresholds are flagged and audited, and higher scores trigger the tool's `on_detect` action. Sanitising widens each span to its sentence or line and replaces it with a bilingual marker; a positive detection without a span fails closed and the content is withheld. Untrusted content that reaches the model is spotlighted [16]: it is wrapped in delimiters whose boundary is derived from a SHA-256 hash of the call identifier, source and content, so the content cannot contain its own closing marker, and a bilingual note tells the model to treat it as data.

The packaged detector currently contains no trained model and runs in rules-only mode, which is also the mode recorded by the latency benchmark (`rules_only`). The F1 scores on the held-out English and Arabic test sets (target 0.85) are produced by running `make detector` after the team has added its labelled data as described in `docs/detector.md`, and Section 4.2.2 reports them. The attack-labelled data, like the attack files of Mirsad-Bench (`bench/scenarios/attacks/README.md`), is written by the team (Section 3.6.4).

3.3.7 Database design

Persistence uses SQLAlchemy 2.0 with typed declarative mappings and only portable column types, so the same schema runs on SQLite in development and PostgreSQL 16 in deployment. SQLite connections enable foreign keys and write-ahead logging, so dashboard readers never block the audit writer. Timestamps are UNIX epoch seconds stored as floats, matching the Pydantic models. Figure 11 shows the six tables.

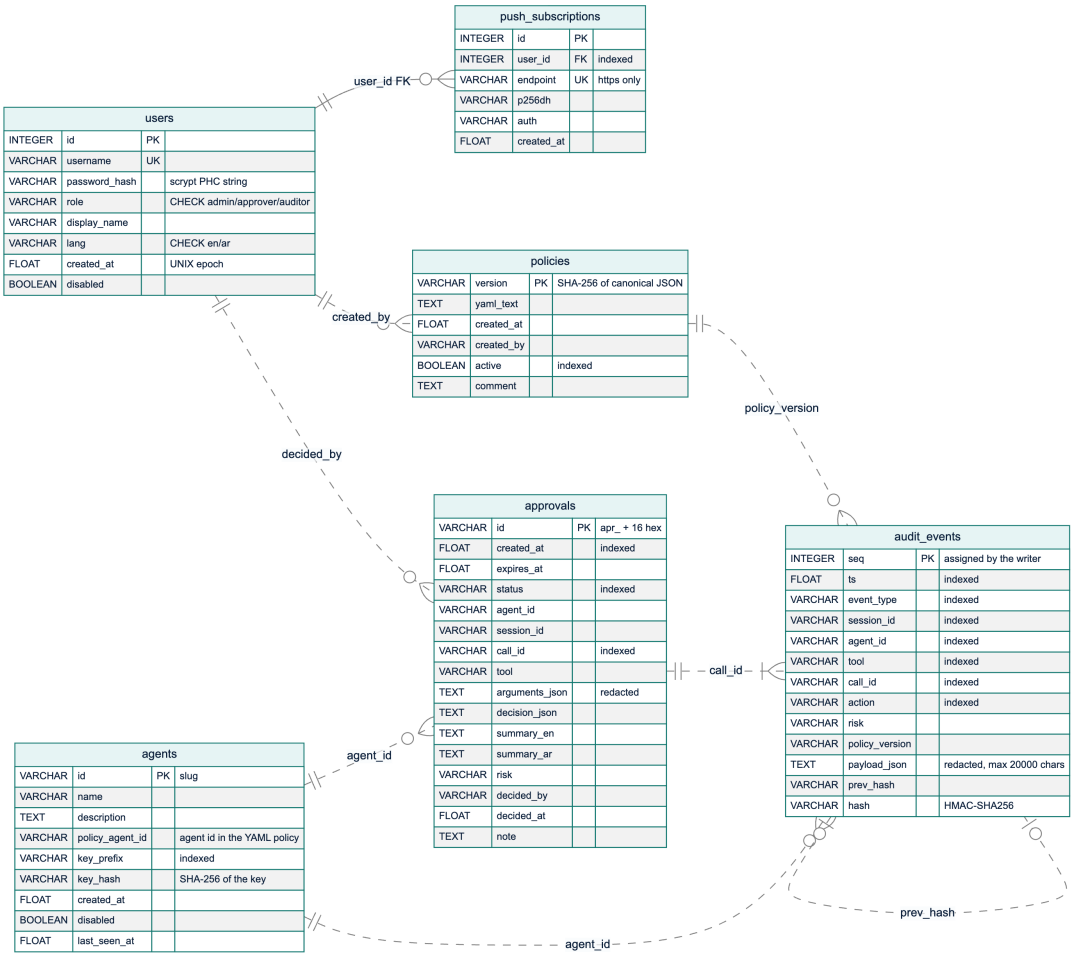


Figure 11: Entity-relationship diagram of the six tables. The solid line is the only enforced foreign key; dashed lines are logical references by value

Only `push_subscriptions.user_id` is an enforced foreign key (to `users.id`, with cascading delete). The other relationships are logical references by value: `audit_events.agent_id` and `approvals.agent_id` hold an agent's policy identifier; `approvals.decided_by` and `policies.created_by` hold usernames; `audit_events.policy_version` holds a `policies.version`; an approval and the audit events of its call share the `call_id`; and each event's `prev_hash` equals the previous event's hash. These references are deliberately not enforced: the audit log is append-only evidence that must outlive the deletion or renaming of what it mentions, and its hashes cover values exactly as written, so a cascading delete or a blocked insert would destroy or prevent evidence.

The management tables are in third normal form. Three columns hold JSON documents by design (`audit_events.payload_json`, `approvals.decision_json`, `approvals.arguments_json`): their shape depends on the event type and they must be stored verbatim to be hashed and replayed, while frequently filtered fields are promoted

to indexed columns. An approval copies the agent, tool and arguments of its call, so the record shows exactly what the approver saw. Table 6 is the table dictionary.

Table 6: Data dictionary of the database tables

Table	Purpose	Key and constraints	Written by / read by
`users`	People who sign in	`id` primary key; `username` unique; role restricted to admin, approver, auditor; language to en, ar	Administrators / authentication, user management
`push_subscriptions`	Browser push endpoints per user	`id` primary key; `endpoint` unique (HTTPS only); `user_id` foreign key with cascade	Approver PWA / Web Push delivery
`agents`	Agent identities and key hashes	`id` (slug) primary key; `key_prefix` indexed; only the SHA-256 hash of the key is stored	Administrators / agent authentication
`policies`	Every policy version stored	`version` (SHA-256) primary key; `active` indexed	Policy activation / gateway start-up, replay, administrators, auditors
`audit_events`	Hash-chained log of decisions, inspections, approvals, administrative changes and logins	`seq` primary key assigned by the writer (no auto-increment); seven indexes	`AuditLog.append` only / administrators, auditors, statistics, replay
`approvals`	Approval requests and outcomes	`id` primary key; indexes on `status`, `created_at`, `call_id`	Approval service / approvers, administrators, waiting calls

Hash-chain design. Each event is a link in a chain built from HMAC-SHA-256 [17], following the hash linking of Haber and Stornetta [18] and the secure audit log of Schneier and Kelsey [19]:

```
hash(n)      = HMAC-SHA256(K_audit, prev_hash(n) + "|" + canonical_json(event(n) without
"hash"))
prev_hash(n) = hash(n-1),   prev_hash(1) = "0" * 64
K_audit      = HMAC-SHA256(secret_key, "mirsad/audit-hmac/v1")
```

Canonical JSON uses sorted keys, no whitespace and UTF-8, so Arabic text is hashed as written. The hashed body contains every stored column, including `seq` and the indexed columns, so editing any field breaks the chain, and the audit key is separate from the JWT signing key although both derive from the server secret. Before hashing, `redact()` removes credentials and masks Saudi national ID and Iqama numbers, IBANs and card numbers to their last four characters (Section 3.6.1), so secrets never enter the chain; payloads are limited to 20,000 characters while keeping their structure. There is one writer per process: under a lock it reads the head and inserts `seq = head + 1` in the same transaction, and if another process takes that number first, the primary key rejects the insert and the writer retries.

Verification walks the table in sequence order and reports the first broken event: a gap in `seq` reveals a deletion, a `prev_hash` that does not link reveals an insertion or reordering, and an HMAC mismatch reveals a modification or a hash forged without the key. The same function verifies an unfiltered JSON export offline. A rewrite by someone holding the key, and truncation of the newest events, cannot be detected from the chain alone; for both, `AuditLog.anchor()` returns the current head to be stored outside the database, and verifying against an earlier anchor detects them. Because enforced calls are refused when the append fails, no action can happen without an event.

3.3.8 Security design and threat model

A firewall concentrates trust, so the gateway itself is a target. The threat model of Section 3.2 was analysed by applying STRIDE to each element of the data-flow diagram [20], [21]. Figure 12 shows the five trust boundaries: untrusted internet content (A), the agent runtime, whose model may be hijacked (B), the gateway (C), the enterprise systems (D) and the human approvers (E). Table 7 lists the main threats, the mitigation in the design and the test that verifies it. Tests ST-01 to ST-23 are in `tests/security/test_gateway_security.py`; other entries name the test modules of the component concerned.

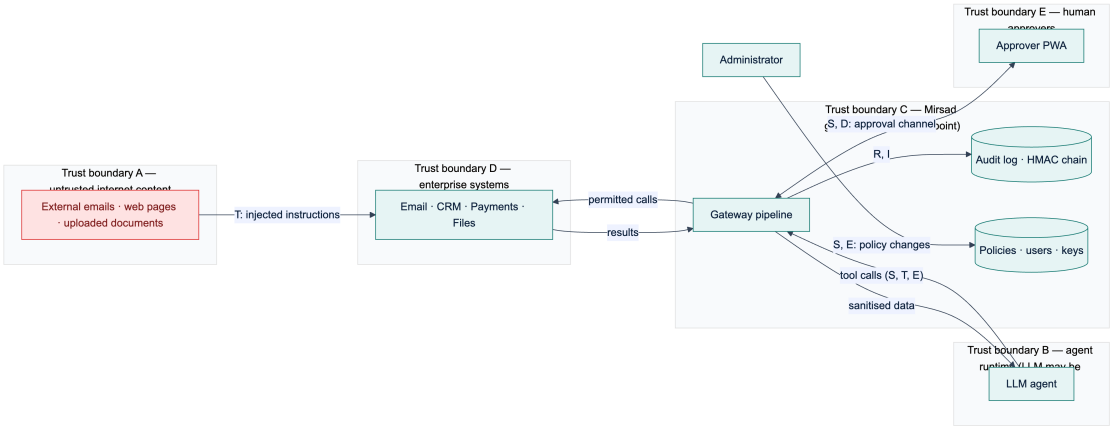


Figure 12: Threat model with trust boundaries A to E and the STRIDE categories of each data flow

Table 7: STRIDE analysis of the gateway

	Threat	Element	Mitigation	Verified by
S	Tools called with a guessed, stolen or revoked agent key	Agent APIs, `/mcp`	Per-agent keys with 256-bit secrets, stored as SHA-256 hashes, compared in constant time; rotation revokes the old key	ST-04, ST-05
S	Forged, unsigned or expired session	Human APIs	HS256 JWT with pinned algorithm,	ST-01, ST-02

	Threat	Element	Mitigation	Verified by
	token		expiry, issuer and role checks; user re-loaded on every request	
S	Online password guessing	Login	scrypt hashes; 5 failures per username and IP (30 per IP) in 5 minutes return HTTP 429	ST-19
T	Audit records edited or deleted to hide an incident	`audit_events`	HMAC hash chain, verification, external anchors	ST-06, ST-07
T	Injected instructions in a tool result steer the agent	Result path	Detector with sanitise, flag or block; spotlighting; detected injection marks the session	`tests/api/test_gateway.py`, `tests/detector/`
T	A policy that hangs the engine (ReDoS) is activated	Policy engine	Admin-only activation; validator rejects nested ambiguous quantifiers; input bound	ST-08
R	An approver denies having approved an action	Approvals	Decider, time and note stored; first decision final; `approval.decided` audited	ST-09
R	Login attempts leave no trace	Authentication	Failed logins audited with the username, never the password	ST-10
I	A hijacked agent sends customer data or money outside the organisation	Call path	Domain constraints; taint rules on untrusted and confidential flows; approval for payments	`tests/taint/`, `tests/policy/`; attack success rate pending (Chapter 4)
I	A URL tool is used to reach internal services (SSRF)	`fetch_url` policy	URL pattern constraints in the default policy	ST-23
I	Secrets in tool arguments leak into the audit log or exports	Audit log	`redact()` before hashing and storage	ST-11
I	Hashes, keys, internal errors or files leak through the API	API	No hashes in listings; generic 500 responses; static files confined to the web root	ST-12, ST-13, ST-14
I	The dashboard is framed or called from another site	Web front end	Same-origin CSP, frame denial, CORS allow-list, no tokens in URLs	ST-15, ST-16
D	Oversized	API, tools	1 MB body limit;	ST-18, `tests/policy/`

	Threat	Element	Mitigation	Verified by
	requests or call flooding		sliding-window rate limits per agent and tool	
D	No approver answers and the call hangs	Approvals	Expiry after the timeout, treated as denial	`tests/audit/test_approval_service.py`
E	A non-admin role performs administrative actions or alters its role claim	Role checks	Role-based access on every route; signed claim; role re-read from the database	ST-03, ST-20
E	An agent key is used on human APIs, or a user token on agent APIs	Authentication	Separate credential types per route group	ST-21
E	A hijacked or unknown agent calls tools outside its role	Policy engine	Default deny; least-privilege tool discovery	ST-22

Further controls apply across components: every component fails closed (Section 3.3.3); searches use parameterised ORM queries, which ST-17 checks with injection-style input; responses carry security headers (content security policy, `nosniff`, frame denial, referrer policy, HSTS over HTTPS); and the server secret is read from the environment or generated into an owner-only file. The security suite contains 23 security tests (45 test cases); Bandit reported 0 issues over 13,023 lines, and dependency audits found 0 vulnerable Python packages and 0 npm vulnerabilities (Section 4.2.6).

These tests verify the gateway's own defences. Whether the design stops hijacked agents is measured by the reduction in attack success rate on Mirsad-Bench relative to an unprotected agent (target at least 80%; Section 4.2.3), once the team adds its attack files to `bench/scenarios/attacks/` (Section 3.6.4).

3.3.9 Interface design

Mirsad has three kinds of external interface: agent interfaces, a REST and streaming interface for people, and the command line. The agent interfaces differ in who executes the tool, which determines the strength of enforcement (Table 8). In **inline** interfaces Mirsad executes the tool itself, so a denied call cannot run. In **advisory** interfaces the agent runtime executes tools and Mirsad removes or refuses denied calls; this protects against a manipulated model but assumes the agent's own code is honest, so inline interfaces are recommended when the runtime is not trusted.

Table 8: Agent-facing interfaces

Interface	Transport	Tool executed by	Enforcement
MCP proxy	stdio (`mirsad mcp-proxy`) or streamable HTTP at `/mcp`	Mirsad	Inline
Tool invocation	`POST /v1/tools/{tool}/invoke`, `GET /v1/tools`	Mirsad	Inline
OpenAI-compatible proxy	`POST /v1/chat/completions`	Agent runtime	Advisory: denied calls removed from the model's response
Guard API and Python SDK	`POST /v1/guard/tool-call`, `POST /v1/guard/tool-result`; `MirsadClient.guard()`	Agent runtime	Advisory: the SDK raises `MirsadBlocked` before the tool runs

MCP proxy. One MCP server is built per agent. `tools/list` returns only the tools the agent's policy allows, and every `tools/call` goes through `Gateway.execute`. Any refusal or internal failure returns an MCP error result with a bilingual explanation whose reason texts are clipped, so a block message cannot re-inject a long attacker-controlled value into the model's context. Only inspected text is returned; MCP structured content is never forwarded, because the sanitiser does not rewrite it.

OpenAI-compatible proxy. For each request the proxy (1) replaces every `role: tool` message with its inspected version, (2) removes tool definitions the policy does not allow, (3) forwards the request to the configured LLM endpoint without the agent's key, and (4) checks every returned tool call, removing denied calls, appending a bilingual explanation and setting `finish_reason` to `stop` when no call remains. Streaming is rejected, because a tool call must be complete before it can be checked.

REST and streaming API. People authenticate with a JWT from `POST /api/auth/login`; agents use their API key in the `Authorization` or `X-Mirsad-Key` header. Agent keys are rejected on `/api/*` and user tokens on `/v1/*`. Table 9 groups the endpoints. Bodies are JSON, errors have the form `{"detail": ...}`, status codes follow HTTP semantics (401, 403, 404, 409 for a conflicting decision or duplicate, 413 for bodies over 1 MB, 422, 429), and an OpenAPI description is generated from the typed models. Live updates use SSE on `GET /api/stream`, read with `fetch` and an `Authorization` header so that tokens never appear in URLs. The `mirsad` command line covers operations that should not depend on the web interface, such as policy activation, audit verification, export and replay, and agent and user creation.

Table 9: REST API overview

Group	Endpoints	Principal	Purpose
Health and authentication	`GET /healthz`; `POST /api/auth/login`; `GET /api/auth/me`	none; any user	Liveness and version; sign-in;

Group	Endpoints	Principal	Purpose
			current user
Agent gateway	`/v1/guard/tool-call`, `/v1/guard/tool-result`, `/v1/tools`, `/v1/tools/{tool}/invoke`, `/v1/chat/completions`, `/mcp`	agent key	Check, inspect, discover and execute tool calls
Approvals	`GET /api/approvals`, `GET /api/approvals/{id}`, `POST /api/approvals/{id}/decision`	admin, approver	Pending queue, history and decisions
Live updates	`GET /api/stream` (SSE); `GET /api/push/vapid`, `POST /api/push/subscribe`, `POST /api/push/unsubscribe`	any user	Approval and audit events; Web Push subscriptions
Audit	`GET /api/audit/events`, `GET /api/audit/events/{seq}`, `POST /api/audit/verify`, `POST /api/audit/events/{seq}/replay`, `GET /api/audit/export`	admin, auditor	Search, verify, replay, export (JSON or CSV)
Policies	`GET /api/policies`, `GET /api/policies/{version}`, `POST /api/policies/validate`, `PUT /api/policies`, `POST /api/policies/{version}/activate`	admin; auditor read-only	Versions, validation, activation, rollback
Agents and users	`/api/agents` (list, create, update, delete, rotate key); `/api/users` (list, create, update, delete)	admin	Identity management; keys shown only once
Tools, statistics, detector	`GET /api/tools`; `GET /api/stats/overview`, `GET /api/stats/timeseries`; `POST /api/detector/scan`	admin, auditor; any user	Tool profiles; dashboard figures; detector playground
Demonstration	`/api/demo/chat`, `/state`, `/reset`, `/scenarios`, `/scenarios/{id}/apply`	any user, demo mode only	Simulated workspace and demo agent

3.3.10 User-interface design

Mirsad has two front ends built from one React and TypeScript code base: the **administration dashboard** at `/` and the installable **approver PWA** at `/approve`. They share the API client, live-stream reader, translation dictionaries, explanation components and design tokens, and the gateway serves both from its own origin, so a strict content security policy that allows only same-origin scripts holds. The interface follows these principles:

1. **Plain language first, evidence on demand.** A request opens with three short sections, *What the agent wants to do*, *Why Mirsad paused it* and *Where the data came from*, followed by the raw arguments, collapsed by default. This applies the human–AI interaction guidelines of explaining why the system acted and supporting

efficient correction [22]. Only escalated calls reach a human, to limit the alert fatigue and complacency that automation research warns about [23].

2. **Bilingual and right-to-left by design.** Every string exists in English and Modern Standard Arabic with the same keys, which tests enforce. Choosing Arabic sets the document's language and direction, and CSS logical properties mirror the whole layout without separate stylesheets. Identifiers such as tool names, addresses, hashes and YAML stay left-to-right inside isolation elements, and server messages are bilingual at the source (`message_ar`, `summary_ar`).
3. **Accessibility to WCAG 2.2 level AA** [24]. Pages use semantic landmarks; every control has a label; everything works from the keyboard; dialogs trap and restore focus; live regions announce new requests; colour is never the only signal; contrast is checked by a test; reduced-motion preferences are respected; and touch targets in the approver app are at least 44 × 44 pixels.
4. **Safe, deliberate actions.** Approve and Deny open a confirmation sheet that restates the consequence and takes an optional note; the buttons are disabled when a request has expired or the phone is offline; API keys and generated passwords are shown once; and text from agents and tools is always rendered as text, never as HTML.
5. **Speed for the approver.** The target is a decision in under 30 seconds on a phone (NFR4): pending requests are sorted by soonest expiry with a countdown, and a push notification opens the request directly.

The dashboard has ten pages (Overview, Activity and audit, Approvals, Policies, Agents, Users, Tools, Detector playground, Demo console, Settings); the sidebar shows only the pages the signed-in role may use, and the API rejects any request outside that role. Figure 13 shows the audit-event drawer with its hash-chain fields and replay and verification actions, and Figure 14 shows the policy editor in Arabic with the mirrored layout. Figure 15 and Figure 16 show the approver app's pending list in Arabic and a request in English. The screenshots were taken from the running system with the simulated workspace; Appendix C contains the complete set.

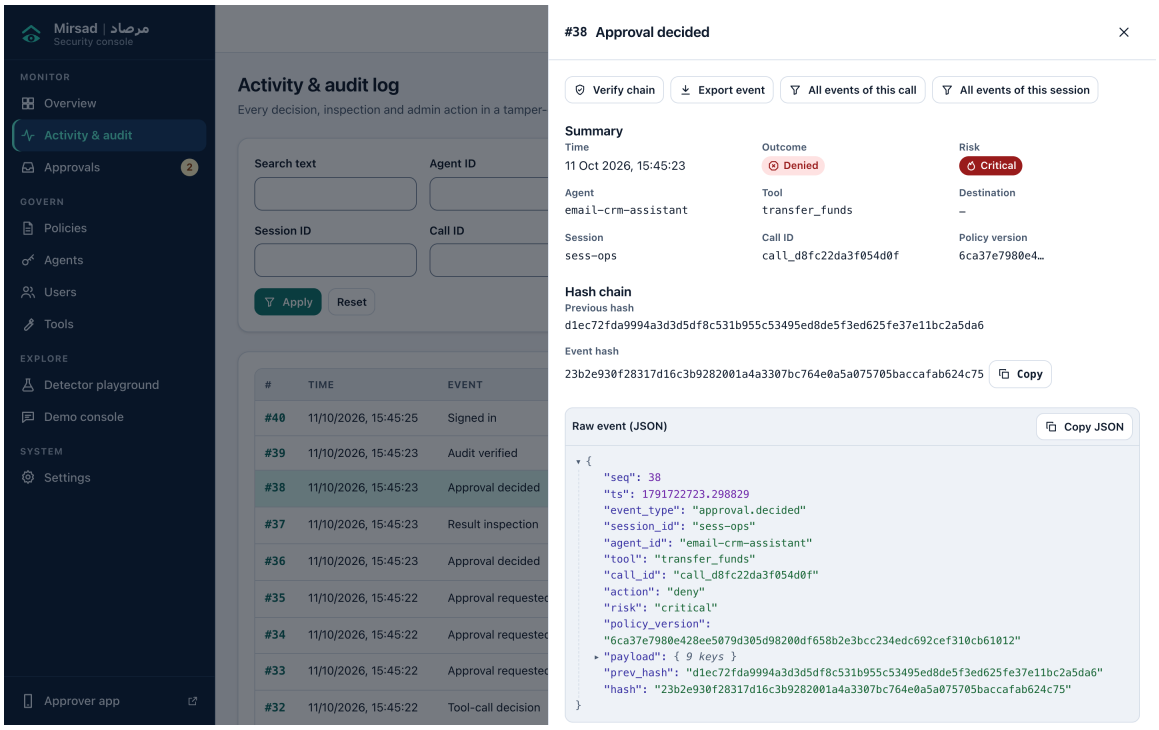


Figure 13: Activity and audit page with the detail drawer of an approval decision: summary, previous hash and event hash, and the raw event

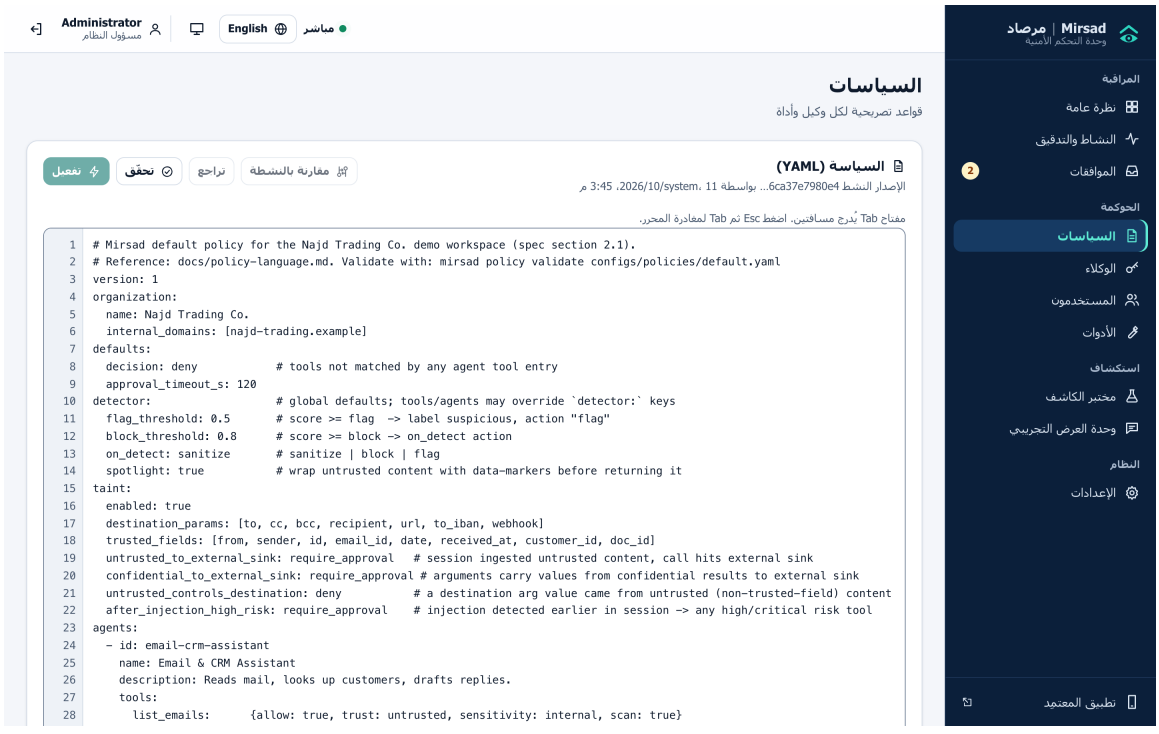


Figure 14: Policy editor in Arabic: the sidebar and controls are mirrored, while the YAML policy stays left-to-right

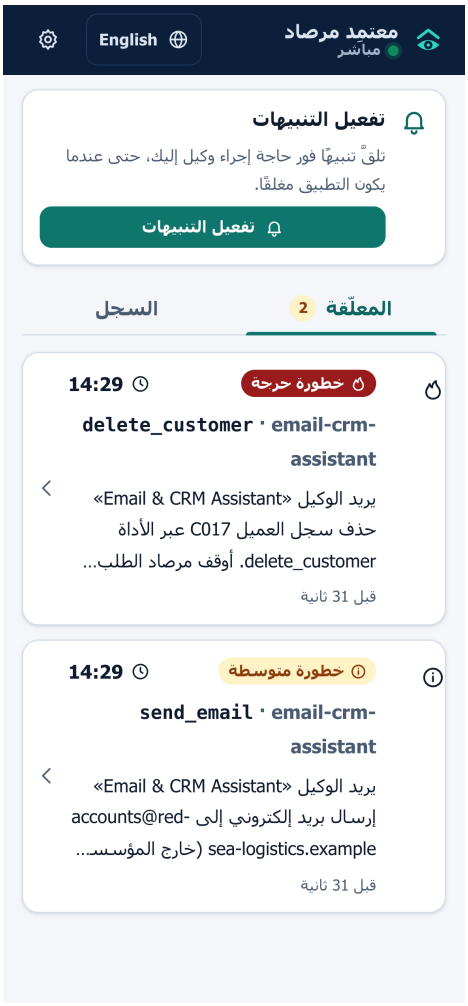


Figure 15: Approver PWA in Arabic: pending requests with risk label, countdown and plain-language summary

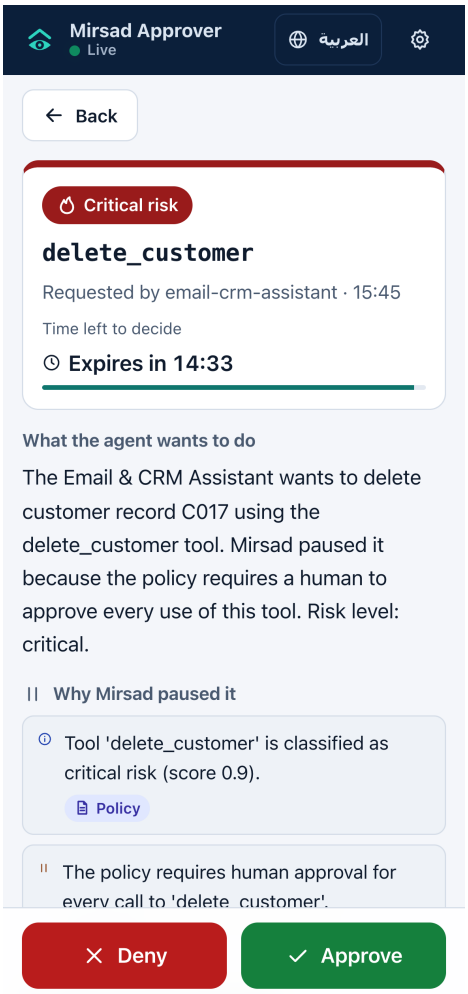


Figure 16: Approver PWA request detail in English: risk, countdown, explanation sections and the Approve and Deny buttons

The approver app is a progressive web app rather than a native application (Section 3.4.5). A web application manifest makes it installable on Android and iOS [25], and a service worker caches the application shell for offline use and receives push messages [26]; decisions always require a connection. Web Push works on Android and, for apps added to the home screen, on iOS 16.4 and later [27]. Service workers and push require HTTPS [28], which the deployment design reflects.

3.3.11 Deployment architecture and technology stack

Figure 17 shows the deployment architecture. The reference deployment is a Docker Compose stack on one server or virtual machine:

- **`mirsad` container.** A multi-stage image in which a Node.js stage builds the web applications and a Python 3.12 stage installs the gateway. It runs as a non-root user, keeps its data in a `/data` volume, exposes port 8700 and has a health check on `/healthz`.

- **`db` container.** PostgreSQL 16 with its own volume and readiness check; the gateway is configured to start only when the database is ready. Development and the benchmark use SQLite instead.
- **Ollama container (optional).** Enabled with the `demo` profile to serve Qwen2.5-7B for the demonstration agent.
- **Upstream tools.** Declared in `configs/upstreams.yaml` as in-process MCP servers, subprocesses over stdio, or remote servers over streamable HTTP.

Agents connect over HTTPS with their API key, and browsers and phones over HTTPS with a JWT. TLS is terminated by a reverse proxy in front of the container, which is not part of the Compose file; HTTPS is mandatory for the approver PWA. The Redis node in the figure marks an optional extension for sharing rate-limit counters between replicas and is not implemented: rate limits are kept in memory per process (Section 3.4.5), which fits the single-node scope. Configuration comes from `MIRSAD_` environment variables or a `.env` file (database URL, server secret, approval timeout, Web Push keys, CORS origins, demonstration switch); a missing secret is generated and stored in an owner-only file.

The Compose files are written but have not been executed, because Docker was not available on the build machine (Section 4.1.1), and the policy and upstream paths must be set explicitly for the `mirsad` service (Appendix E.2.2).

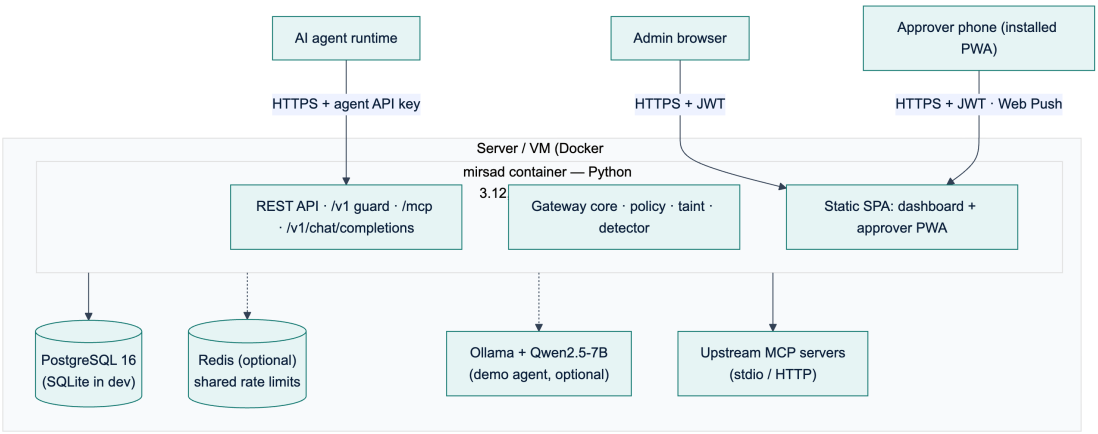


Figure 17: Deployment architecture: the Mirsad container, the database, optional components and the clients

The design targets one node serving at least 50 concurrent agent sessions (NFR11) with a median added latency of at most 200 ms (NFR1); Chapter 4 reports the measured latency (Section 4.2.5). Scaling beyond one node would require moving rate-limit counters and session taint state into a shared store, which is future work. The technologies of each layer and the reasons for choosing them are given in the technology-stack table of Section 3.4.4.

References

[1] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, vol. 63, no. 9, pp. 1278–1308, Sep. 1975, doi: 10.1109/PROC.1975.9939.

[2] E. Rissanen, Ed., "eXtensible Access Control Markup Language (XACML) Version 3.0," OASIS Standard, Jan. 22, 2013. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec '23), Copenhagen, Denmark, 2023, pp. 79-90, doi: 10.1145/3605764.3623985.

[4] E. Debenedetti et al., "Defeating prompt injections by design," arXiv:2503.18813, Mar. 2025 (v2, Jun. 2025).

[5] L. Beurer-Kellner et al., "Design patterns for securing LLM agents against prompt injections," arXiv:2506.08837, Jun. 2025.

[6] T. Shi et al., "Progent: Securing AI agents with privilege control," arXiv:2504.11703, Apr. 2025 (v3, May 2026).

[7] M. Nasr et al., "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," in Proc. 35th USENIX Security Symp. (USENIX Security 26), Baltimore, MD, USA, Aug. 2026, pp. 1467-1486.

[8] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proc. 18th USENIX Secur. Symp., Montreal, QC, Canada, Aug. 2009.

[9] Open Policy Agent, "Policy language," OPA Documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-language.

[10] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," IEEE J. Sel. Areas Commun., vol. 21, no. 1, pp. 5–19, Jan. 2003, doi: 10.1109/JSAC.2002.806121.

[11] J. Newsome and D. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in Proc. 12th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS), San Diego, CA, USA, Feb. 2005.

[12] S. Willison, "The lethal trifecta for AI agents: private data, untrusted content, and external communication," Simon Willison's Weblog, Jun. 16, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/

[13] M. Costa et al., "Securing AI agents with information-flow control," arXiv:2505.23643, May 2025.

[14] F. Wu, E. Cecchetti, and C. Xiao, "System-level defense against indirect prompt injection attacks: An information flow control perspective," arXiv:2409.19091, Sep. 2024.

[15] D. E. Denning, "A lattice model of secure information flow," Commun. ACM, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.

[16] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kıcıman, "Defending against indirect prompt injection attacks with spotlighting," in Proc. Conf. Applied Machine Learning in Information Security (CAMLIS 2024), CEUR Workshop Proceedings, vol. 3920, Arlington,

VA, USA, Oct. 2024, pp. 48-62. Accessed: Oct. 11, 2026. [Online]. Available: https://ceur-ws.org/Vol-3920/paper03.pdf

[17] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.

[18] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," J. Cryptol., vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.

[19] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," ACM Trans. Inf. Syst. Secur., vol. 2, no. 2, pp. 159–176, May 1999, doi: 10.1145/317087.317089.

[20] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: John Wiley & Sons, 2014, ISBN: 978-1-118-80999-0.

[21] Microsoft, "Threats - Microsoft Threat Modeling Tool - Azure," Microsoft Learn. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats.

[22] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-AI interaction," in Proc. CHI Conf. Human Factors Comput. Syst. (CHI '19), Glasgow, U.K., 2019, pp. 1–13, doi: 10.1145/3290605.3300233.

[23] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," IEEE Trans. Syst., Man, Cybern. A, Syst. Humans, vol. 30, no. 3, pp. 286–297, May 2000, doi: 10.1109/3468.844354.

[24] A. Campbell, C. Adams, R. B. Montgomery, M. Cooper, and A. Kirkpatrick, Eds., "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023 (updated Dec. 12, 2024). Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/WCAG22/

[25] M. Cáceres, D. Murphy, and C. Liebel, Eds., "Web Application Manifest," W3C Working Draft, Oct. 9, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/appmanifest/

[26] M. Chintala and Y. Yanagisawa, Eds., "Service Workers Nightly," W3C Candidate Recommendation Draft, Sep. 17, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/service-workers/

[27] B. Eidson and J. Simmons, "Web Push for Web Apps on iOS and iPadOS," WebKit Blog, Feb. 16, 2023. [Online]. Available: https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/. Accessed: Oct. 11, 2026.

[28] MDN contributors, "Service Worker API," MDN Web Docs, Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API. Accessed: Oct. 11, 2026.